

Repaso 11

SORTING - ARBOLES - ORDEN LINEAL



Ordenamiento - Examen 2023-2 P1

- StrandSort es un algoritmo de ordenación que utiliza una secuencia auxiliar para ordenar una secuencia A

```
StrandSort( $A$ ):  
1   $B \leftarrow$  Secuencia vacía  
2   $C \leftarrow$  Secuencia vacía  
3  while  $A$  tiene elementos :  
4       $x \leftarrow$  primer elemento de  $A$   
5      Borrar  $x$  de  $A$  e insertarlo al final de  $B$   
6      foreach  $a$  elemento de  $A$  do  
7          if  $a >$  último elemento de  $B$  :  
8              Borrar  $a$  de  $A$  e insertarlo al final de  $B$   
9       $C \leftarrow \text{Merge}(B, C)$   
10      $B \leftarrow$  Secuencia vacía  
11     return  $C$ 
```

Ordenamiento - Examen 2023-2 P1

- a) Demuestre que StrandSort termina
- b) Como Merge es correcto, podemos demostrar que StrandSort ordena sin usar inducción. Demuestre que el algoritmo es correcto. *Hint: ¿Qué cumple B ?*
- c) Determina si StrandSort tiene un mejor y/o peor caso. Justifique
- d) Determine la complejidad de tiempo y memoria de los casos identificados en c)

Ordenamiento - Solución

a) Demuestre que StrandSort termina

```
StrandSort( $A$ ):  
1    $B \leftarrow$  Secuencia vacía  
2    $C \leftarrow$  Secuencia vacía  
3   while  $A$  tiene elementos :  
4        $x \leftarrow$  primer elemento de  $A$   
5       Borrar  $x$  de  $A$  e insertarlo al final de  $B$   
6       foreach  $a$  elemento de  $A$  do  
7           if  $a >$  último elemento de  $B$  :  
8               Borrar  $a$  de  $A$  e insertarlo al final de  $B$   
9        $C \leftarrow$  Merge( $B, C$ )  
10       $B \leftarrow$  Secuencia vacía  
11      return  $C$ 
```

Ordenamiento - Solución

a) Demuestre que StrandSort termina

Observemos que las líneas 4 y 5 seleccionan y extraen un elemento de A en cada iteración del **while**. Además el **foreach** de la línea 6 itera sobre los elementos restantes de A . Como A es finita, el bloque **while** se itera a lo más $|A|$ veces. Como las operaciones de inserción, extracción y Merge, son finitas, el algoritmo termina.



Ordenamiento - Solución

b) Como Merge es correcto, podemos demostrar que StrandSort ordena sin usar inducción. Demuestre que el algoritmo es correcto. *Hint: ¿Qué cumple B?*

```
StrandSort(A):
1   B ← Secuencia vacía
2   C ← Secuencia vacía
3   while A tiene elementos :
4       x ← primer elemento de A
5       Borrar x de A e insertarlo al final de B
6       foreach a elemento de A do
7           if a > último elemento de B :
8               Borrar a de A e insertarlo al final de B
9       C ← Merge(B, C)
10  B ← Secuencia vacía
11  return C
```

Ordenamiento - Solución

b) Como Merge es correcto, podemos demostrar que StrandSort ordena sin usar inducción. Demuestre que el algoritmo es correcto. *Hint: ¿Qué cumple B?*

Dado que Merge es correcto, sabemos que cuando recibe secuencias ordenadas, produce una nueva secuencia ordenada. Consideremos primero, que todo elemento de A es extraído en alguna iteración del while y es agregado a B solo si es mayor que todos los elementos en dicha secuencia. Con esto B esta ordenada.

Además, al sobrescribir C con el resultado del Merge, nos aseguran de que está cada elemento de A al terminar el algoritmo, y dado que Merge ordena, sabemos que C esta ordenada y concluimos que StrandSort ordena.

Ordenamiento - Solución

c) Determina si StrandSort tiene un mejor y/o peor caso. Justifique

```
StrandSort(A):  
1   $B \leftarrow$  Secuencia vacía  
2   $C \leftarrow$  Secuencia vacía  
3  while  $A$  tiene elementos :  
4       $x \leftarrow$  primer elemento de  $A$   
5      Borrar  $x$  de  $A$  e insertarlo al final de  $B$   
6      foreach  $a$  elemento de  $A$  do  
7          if  $a >$  último elemento de  $B$  :  
8              Borrar  $a$  de  $A$  e insertarlo al final de  $B$   
9       $C \leftarrow \text{Merge}(B, C)$   
10      $B \leftarrow$  Secuencia vacía  
11     return  $C$ 
```


Ordenamiento - Solución

c) Determina si StrandSort tiene un mejor y/o peor caso. Justifique

El mejor caso del algoritmo corresponde a una secuencia A ordenada. En tal caso, solo ocurre una iteración del while pues el bloque foreach extrae todos los elementos de A consecutivamente. El peor caso corresponde a una secuencia de orden decreciente, en cuyo caso, hay n iteraciones del while cuando A tiene n elementos



Ordenamiento - Solución

d) Determine la complejidad de tiempo y memoria de los casos identificados en c)

```
StrandSort(A):  
1   B ← Secuencia vacía  
2   C ← Secuencia vacía  
3   while A tiene elementos :  
4       x ← primer elemento de A  
5       Borrar x de A e insertarlo al final de B  
6       foreach a elemento de A do  
7           if a > último elemento de B :  
8               Borrar a de A e insertarlo al final de B  
9       C ← Merge(B, C)  
10      B ← Secuencia vacía  
11      return C
```

Ordenamiento - Solución

d) Determine la complejidad de tiempo y memoria de los casos identificados en c)

Si consideramos la implementación de Merge que usa memoria lineal, el algoritmo toma tiempo $O(n)$ y memoria $O(n)$ en el mejor caso, pues se ejecuta una iteración del **while** y los n pasos del bloque **foreach**. Por su parte, en el peor caso el tiempo es $O(n^2)$ y memoria $O(n)$, pues ocurren n iteraciones del **while** y los **foreach** reducen su largo en 1 (similar a algoritmos estudiados en clases)

Ordenamiento – QuickSort y MergeSort

- a) ¿Es correcto que si la rutina Partition siempre realiza el mínimo número de intercambios posibles, se garantiza que QuickSort ejecuta en el menor tiempo posible (es decir, está en el mejor caso)? Argumente.
- b) Escriba el pseudocódigo de una versión inestable de MergeSort. Luego diga cómo transformarlo en estable



Ordenamiento – QuickSort y MergeSort

- a) **¿Es correcto que si la rutina Partition siempre realiza el mínimo número de intercambios posibles, se garantiza que QuickSort ejecuta en el menor tiempo posible (es decir, está en el mejor caso)? Argumente.**

Falso. Los peores casos de QuickSort son aquellos en los que ocurren muchas particiones desbalanceadas. Una partición desbalanceada es cuando el pivote, luego de Partition, queda al inicio o final del arreglo, y esto no tiene relación alguna con el número de intercambios realizados (puede haber una partición desbalanceada con muchos intercambios así como una sin intercambio). Si ocurren muchas particiones desbalanceadas en QuickSort, entonces cada llamada recursiva procesará un arreglo de $n - 1$ elementos (arreglo original menos el pivote), por lo que se llamará n veces hasta llegar a un arreglo de largo 1.

Ordenamiento – QuickSort y MergeSort

b) Escriba el pseudocódigo de una versión inestable de MergeSort. Luego diga cómo transformarlo en estable

Un algoritmo estable es aquel que mantiene el orden del input original cuando el algoritmo de comparación no es capaz de distinguir dos o más elementos. Va a garantizar que cuando dos objetos son iguales, los ordenará de la misma forma que estaban originalmente

Ordenamiento – QuickSort y MergeSort

En el algoritmo de Merge, se comparan los objetos del arreglo de la derecha y de la izquierda, poniendo primero en el arreglo original el que sea más pequeño.

Ahora, si cambiamos la comparación de $\leq$ por solo $<$ en el algoritmo, podemos obtener una versión inestable de MergeSort, pues cuando hayan dos objetos iguales no entrará en la primera condición del if y entrará en el else, que es donde se ubica el objeto del arreglo de la derecha primero.



Ordenamiento – QuickSort y MergeSort

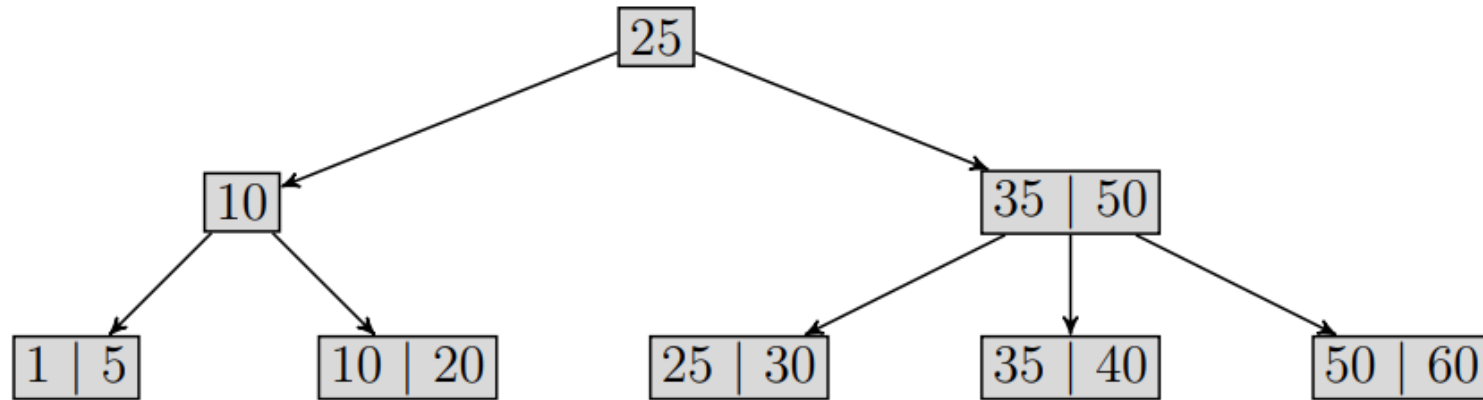
El cambio se encuentra en la línea 9 del siguiente pseudocódigo:

```
1: function MERGESORT( $A, p, q, r$ )
2:    $i \leftarrow 0, j \leftarrow 0$ 
3:    $L$  nuevo arreglo de tamaño  $q - p + 2$ 
4:    $R$  nuevo arreglo de tamaño  $r - q + 2$ 
5:    $L[0 \dots q - p] \leftarrow A[p \dots q]$ 
6:    $R[0 \dots r - q - 1] \leftarrow A[q + 1 \dots r]$ 
7:    $L[q - p + 1] \leftarrow R[r - q] \leftarrow \infty$ 
8:   for  $k \leftarrow p$  to  $r$  do
9:     if  $L[i] < R[j]$  then
10:       $A[k] \leftarrow L[i]$ 
11:       $i++ = 1$ 
12:     else
13:       $A[k] \leftarrow R[j]$ 
14:       $j++ = 1$ 
15:     end if
16:   end for
17: end function
```

Para hacerlo estable, se cambia $<$ por $\leq$ en la línea 9 (**1 punto** por decir cómo hacerlo estable).

Árboles ~ 11 2025-1

Considera la siguiente variante de un árbol 2-3 en donde todas las hojas son nodos de tipo 3-nodo, y las dos claves almacenadas en una hoja representan un intervalo. Por ejemplo, en la siguiente figura están representados los intervalos [1-5], [10-20], [25-30], [35-40], y [50-60].



Árboles ~ 11 2025-1

El resto de los nodos (los que no son hojas) pueden ser de tipo 2-nodo o 3-nodo y contienen algunas de las claves iguales a las contenidas en las hojas. Es decir, una clave que está en una hoja puede estar también en un nodo no hoja. Por ejemplo, la clave 10 está contenida en el 2-nodo [10] y en su hijo [10-20] (que es una hoja), mientras que la clave 7, al no estar contenida en ninguna hoja, tampoco estará en un nodo no hoja. Todos los intervalos almacenados en los nodos hoja son disjuntos, es decir $\forall X, Y \in \text{hojas}, X \cap Y = \emptyset$ (no se solapan). La figura mostrada anteriormente ejemplifica la estructura posible para los intervalos ya indicados.

Árboles ~ 11 2025-1

- a) Usando la estrategia de dividir para conquistar, escriba en pseudocódigo la función f $\text{ind}(k) \rightarrow [k_i, k_f]$ que busque un valor k en el árbol y entregue el 3-nodo que lo contiene i.e. $k_i \leq k \leq k_f$ si existe y null si no. P. ej. $f \text{ ind}(4) \rightarrow [1, 5]$ y $f \text{ ind}(43) \rightarrow \text{null}$
- b) ¿Qué modificaciones introduciría en la estructura anterior para que dadas dos claves k, j , con $k < j$ permita entregar, en $O(n)$, la lista de intervalos que contienen desde la clave k (primer intervalo) hasta la clave j (último intervalo). P. ej. $k = 4$ y $j = 39$ entrega el resultado $[1-5], [10-20], [25-30], [35-40]$ Hint: Para modificar la estructura puede agregar punteros a los nodos, punteros entre nodos, o parámetros extra a los nodos.
- c) Para la estructura modificada por usted en b) escriba en pseudocódigo la función $\text{Range}(k, j) \rightarrow [k_a, k_b], \dots, [k_z, k_t]$ descrita anteriormente en b). Para ello hay 3 casos

Árboles ~ 11 2025-1

- a) Usando la estrategia de dividir para conquistar, escriba en pseudocódigo la función $\text{find}(k) \rightarrow [k_i, k_f]$ que busque un valor k en el árbol y entregue el 3-nodo que lo contiene i.e. $k_i \leq k \leq k_f$ si existe y null si no. P. ej. $\text{find}(4) \rightarrow [1, 5]$ y $\text{find}(43) \rightarrow \text{null}$

A diferencia de un árbol 2-3 original donde la búsqueda se puede detener en un nodo no hoja cuando se encuentra la clave, en esta versión se debe continuar la búsqueda hasta la hoja

Árboles ~ 11 2025-1

Algorithm 1: $\text{find}(k, t)$

```
1 begin
2   Condiciones de término
3   if  $t = \text{hoja}$  and  $k_1 \leq k \leq k_2$  then
4     return  $[k_1, k_2]$ 
5   else
6     return null
7   recorrido por el árbol
8   if  $t$  es nodo tipo 2 then
9     if  $k \leq k_1$  then
10      find( $k, t.\text{left}$ )
11      Siga buscando en rama izquierda
12    else
13      find( $k, t.\text{right}$ )
14      Siga buscando en rama derecha
15   else
16     if  $k \leq k_1$  then
17      find( $k, t.\text{left}$ )
18      // Siga buscando en rama izquierda
19    else
20      if  $k_1 \leq k \leq k_2$  then
21        find( $k, t.\text{center}$ )
22        // Siga buscando en rama central
23      else
24        find( $k, t.\text{right}$ )
25        // Siga buscando en rama derecha
```

Árboles ~ 11 2025-1

b) ¿Qué modificaciones introduciría en la estructura anterior para que dadas dos claves k, j , con $k < j$ permita entregar, en $O(n)$, la lista de intervalos que contienen desde la clave k (primer intervalo) hasta la clave j (último intervalo). P. ej. $k = 4$ y $j = 39$ entrega el resultado $[1-5], [10-20], [25-30], [35-40]$ Hint: Para modificar la estructura puede agregar punteros a los nodos, punteros entre nodos, o parámetros extra a los nodos.

Se unen las hojas como una lista ligada simple o doble. La búsqueda con find es $O(\log(n))$ y el recorrido $O(n)$

Árboles ~ 11 2025-1

c) Para la estructura modificada por usted en b) escriba en pseudocódigo la función $\text{Range}(k, j) \rightarrow [ka, kb], \dots, [kz, kt]$ descrita anteriormente en b). Para ello hay 3 casos

- **k y j están contenidos en dos intervalos, p. ej. $\text{Range}(4, 39) \rightarrow [1-5], [10-20], [25-30], [35-40]$**
- **k y j están en el mismo intervalo, p. ej. $\text{Range}(10, 19) \rightarrow [10-20]$**
- **k o j, o ambos, no están en los intervalos, p. ej. $\text{Range}(1, 22) \rightarrow \text{null}$**

Árboles ~ 11 2025-1

Range(k, j):

nodolInicio $\leftarrow$ find(k)

if nodolInicio = null:

return null // k no pertenece a ningún intervalo

[k1, k2] $\leftarrow$ intervalo de nodolInicio

if $j \leq k2$:

// caso 1: k y j están en el mismo intervalo

return [[k1, k2]]

...

...

// caso 2: k y j en distintos intervalos

resultado $\leftarrow$ [[k1, k2]] // agregamos el primer intervalo

nodo $\leftarrow$ nodolInicio.next // usamos el puntero a la siguiente hoja

while nodo $\neq$ null:

[a, b] $\leftarrow$ intervalo de nodo

resultado.agregar([a, b])

if $j \leq b$:

// ya encontramos el intervalo que contiene a j

return resultado

nodo $\leftarrow$ nodo.next

// caso 3: llegamos al final y no encontramos j

return null

Árboles ~ 11 2025-1

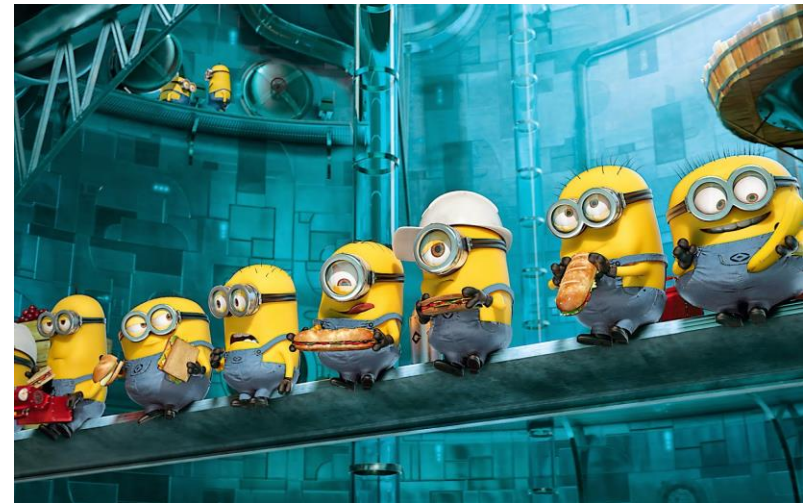
Para la estructura modificada por usted en b) escriba en pseudocódigo la función $\text{Range}(k, j) \rightarrow [k_a, k_b], \dots, [k_z, k_t]$ descrita anteriormente en b)

Algorithm 2: $\text{Range}(k, j)$

```
1 if find(k) = null then
2   | Return null No hay valor de inicio
3 else
4   | if  $j \leq k_2$  then
5     | Return  $[k_1, k_2]$  El rango está en un solo nodo
6   | else
7     |  $r \leftarrow [k_a, k_b]$  primer intervalo;
8     | Recorra la lista en (t hasta t=null y j es mayor que  $k_i$  agregue la hoja a r;
9     | return r;
```

Orden Lineal

Entregue un algoritmo que, dados n enteros en el rango 0 a k , los preprocesa en tiempo $O(n + k)$ y luego responde cualquier consulta acerca de cuántos de los n enteros están en el rango $[a \dots b]$ en tiempo $O(1)$



Orden Lineal - Solución

Entregue un algoritmo que, dados n enteros en el rango 0 a k , los preprocesa en tiempo $O(n + k)$ y luego responde cualquier consulta acerca de cuántos de los n enteros están en el rango $[a \dots b]$ en tiempo $O(1)$

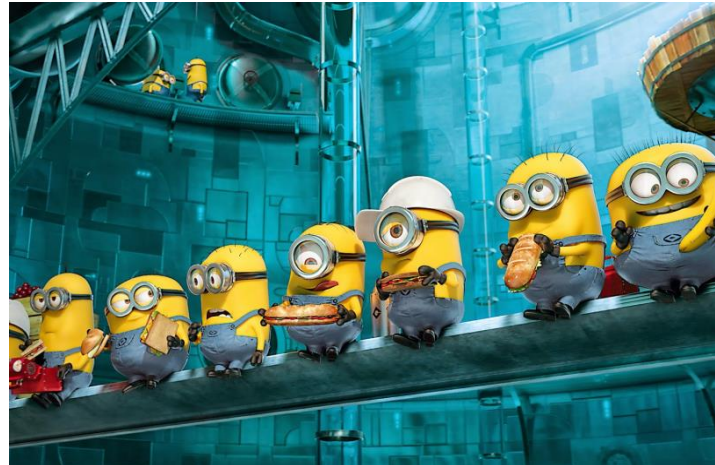
Suponemos que los n enteros vienen en un arreglo A . La idea es usar la primera parte de CountingSort, es decir, los primeros tres ciclos **for**, que son los que arman el arreglo C

Orden Lineal - Solución

CountingSort (A, k):

```
1   $B[0 \dots n-1] \leftarrow$  arreglo vacío de  $n$  celdas
2   $C[0 \dots k] \leftarrow$  arreglo vacío de  $k+1$  celdas
3  for  $i = 0 \dots k$  :
4       $C[i] \leftarrow 0$ 
5  for  $j = 0 \dots n-1$  :
6       $C[A[j]] \leftarrow C[A[j]] + 1$ 
7  for  $p = 1 \dots k$  :
8       $C[p] \leftarrow C[p] + C[p-1]$ 
9  for  $r = n-1 \dots 0$  :
10      $B[C[A[r]] - 1] \leftarrow A[r]$ 
11      $C[A[r]] \leftarrow C[A[r]] - 1$ 
12 return  $B$ 
```

Con esto, el número de enteros en el rango $[a \dots b]$ es $C[b] - C[a - 1]$



¡Mucho Éxito en la 11!

